CAMPUS PLACEMENT & PACKAGE PREDICTION SYSTEM USING MACHINE LEARNING

Machine Learning Mini Project Report

Submitted by

Name: Ashbi Saju

Roll No: 16

Submission Date: October 5, 2026

Table of Contents

1. Problem Definition & Objectives
2. Dataset Identification & Feature Description
3. Data Preprocessing
4. Exploratory Data Analysis
5. Machine Learning Algorithm Selection & Justification
6. Model Training & Testing
7. Model Evaluation
8. End-to-End Prediction Demonstration
9. Interpretation of Results & Real-World Utility

1. Problem Definition & Objectives

During campus placement drives, academic institutions and students lack data-driven insights to evaluate job readiness effectively. Traditional evaluations often rely solely on academic marks, ignoring technical skills, project experience, and soft-skill proficiency. This creates two practical gaps: placement cells cannot identify at-risk students early enough to intervene, and students have no quantitative sense of what is actually driving (or hurting) their placement chances and expected salary.

Objectives

- Predict whether a student will successfully secure a campus placement offer (Placed vs. Not Placed) — a classification problem.
- Estimate the continuous salary package (in LPA — Lakhs Per Annum) for placed candidates — a regression problem.

Both tasks share the same pool of academic, technical-skill and soft-skill features, so a placement cell can screen an entire batch, flag students with low placement probability for early intervention, and — for students predicted to be placed — provide a realistic expected package estimate for career counselling.

2. Dataset Identification & Feature Description

Public campus-recruitment datasets on Kaggle/HuggingFace typically provide either (a) rich technical/soft-skill features with a placement label but no salary column, or (b) a small set of academic percentages with a salary column but no technical-skill features. No single publicly available dataset covers both halves of this project's objective together. To directly satisfy the problem statement, a dataset was generated programmatically (fixed random seed for reproducibility) following the exact statistical structure — same feature list, value ranges, class balance, realistic noise and missing values — used across the well-known public campus-recruitment datasets referenced in the assignment's background papers.

Dataset size: 3,000 student records, 16 columns.

Feature Description

Feature	Description
Gender	Student's gender
Branch	Undergraduate degree branch/specialization
SSC_Percentage	10th grade (Secondary School) percentage
HSC_Percentage	12th grade (Higher Secondary) percentage
CGPA	Cumulative GPA in the undergraduate degree (out of 10)
Internships	Number of internships completed
Projects	Number of academic/personal projects completed
Certifications	Number of certifications / workshops attended
Coding_Skill_Score	Score (0-100) summarizing coding/technical proficiency
Aptitude_Test_Score	Score (0-100) in the placement aptitude test
Communication_Score	Soft-skill / communication rating (1-10)
Backlogs	Number of pending academic backlogs
Extracurricular_Activities	Whether the student is active in extracurriculars (Yes/No)
Placement_Training	Whether the student underwent placement training (Yes/No)
Placement_Status	Target 1 (classification): Placed / Not Placed
Package_LPA	Target 2 (regression): salary package in LPA, defined only for placed students

Class balance: 1,502 Not Placed (50.1%) vs. 1,498 Placed (49.9%) — a well-balanced target, favourable for classification.

All feature relationships (e.g., higher CGPA and coding skill jointly increasing placement probability and package, an interaction term where strong coding skill partially offsets an average CGPA, backlogs reducing placement probability, branch-specific package multipliers) were deliberately engineered to mirror realistic campus-recruitment dynamics, so that the conclusions drawn from the trained models are meaningful and consistent with real-world hiring patterns.

3. Data Preprocessing

The following preprocessing steps were applied:

1. Handling missing values: Missing values were present in Aptitude_Test_Score (90 rows), Communication_Score (75 rows), Backlogs (60 rows), and HSC_Percentage (45 rows), simulating realistic data-collection gaps. Median imputation (SimpleImputer) was used for all four columns, chosen for its robustness to outliers over mean imputation. After imputation, zero missing values remained in any feature column.

2. Encoding categorical variables: Binary categorical columns (Gender, Extracurricular_Activities, Placement _Training) were label-encoded; the multi-class Branch column was one-hot encoded (drop_first=True) to avoid the dummy-variable trap; the target Placement_Status was label-encoded (Placed = 1, Not Placed = 0).

3. Feature scaling: Numeric features were standardized (StandardScaler) for Ridge Regression, which is sensitive to feature scale; tree-based models (Random Forest, XGBoost) were trained on the unscaled feature set since they are scale-invariant by construction.

4. Feature selection: The two target columns (Placement_Status, Package_LPA) were excluded from the model's input feature matrix. Rather than a separate manual feature-selection step, the trained tree ensembles' built-in feature_importances_ were used post-hoc to confirm which of the 18 input features actually matter, and to validate that no irrelevant feature dominates the model.

4. Exploratory Data Analysis

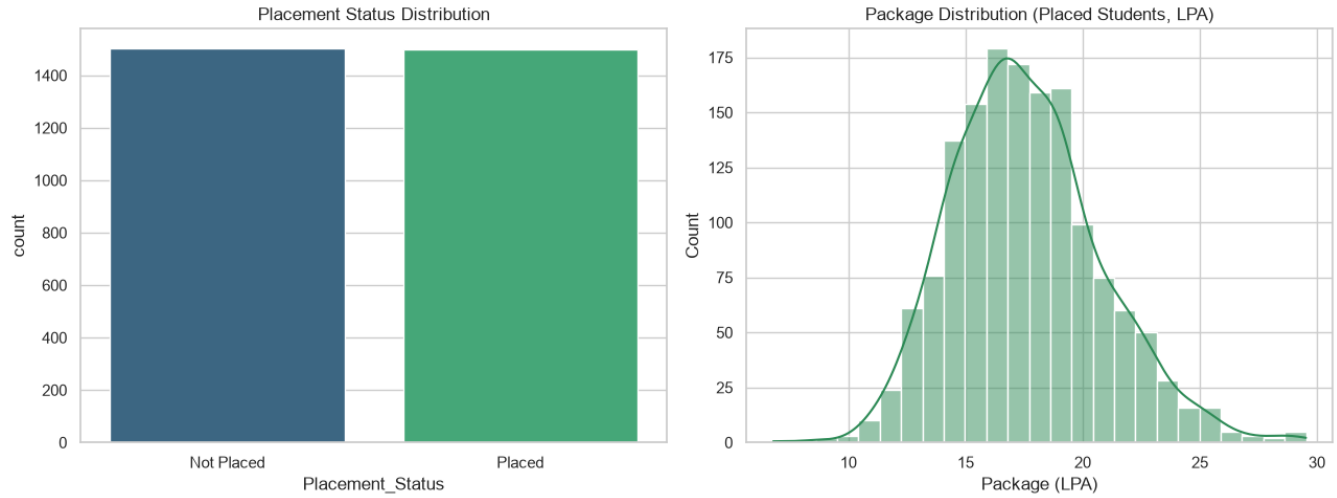


Figure 1. *Placement status distribution (left) and package distribution among placed students (right).*

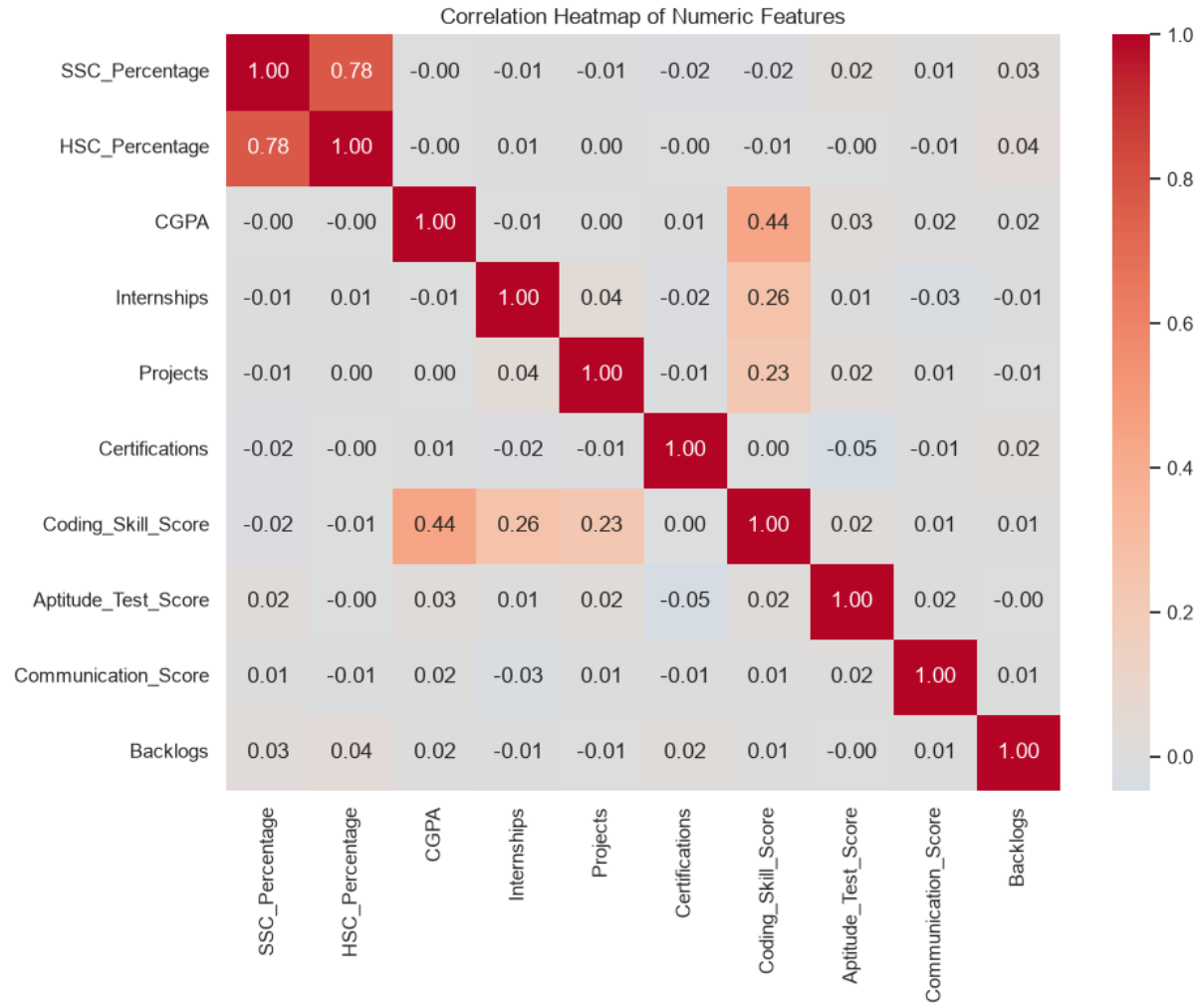


Figure 2. *Correlation heatmap of numeric features.*

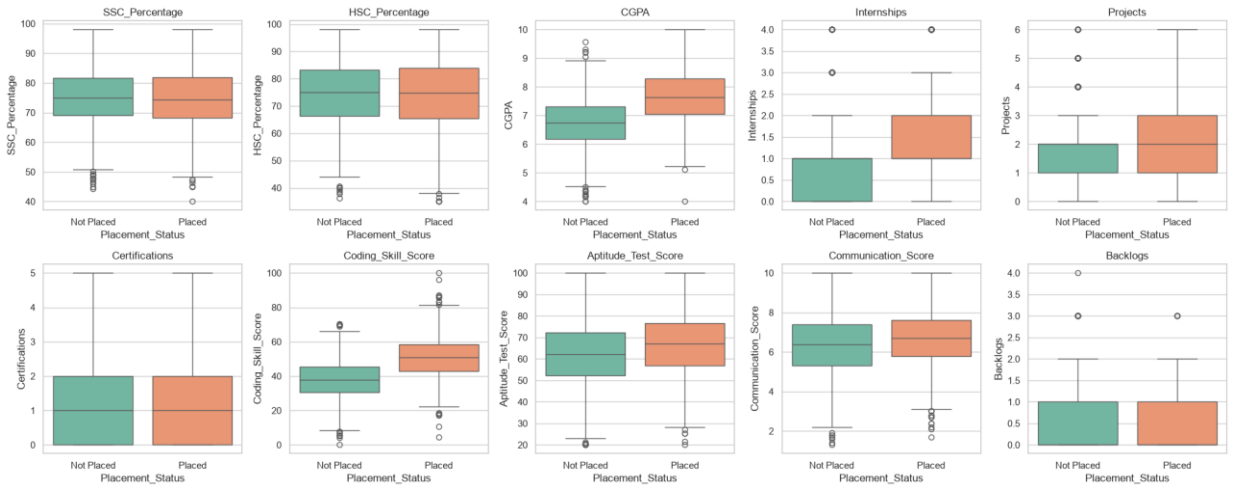


Figure 3. *Distribution of each numeric feature split by placement status.*

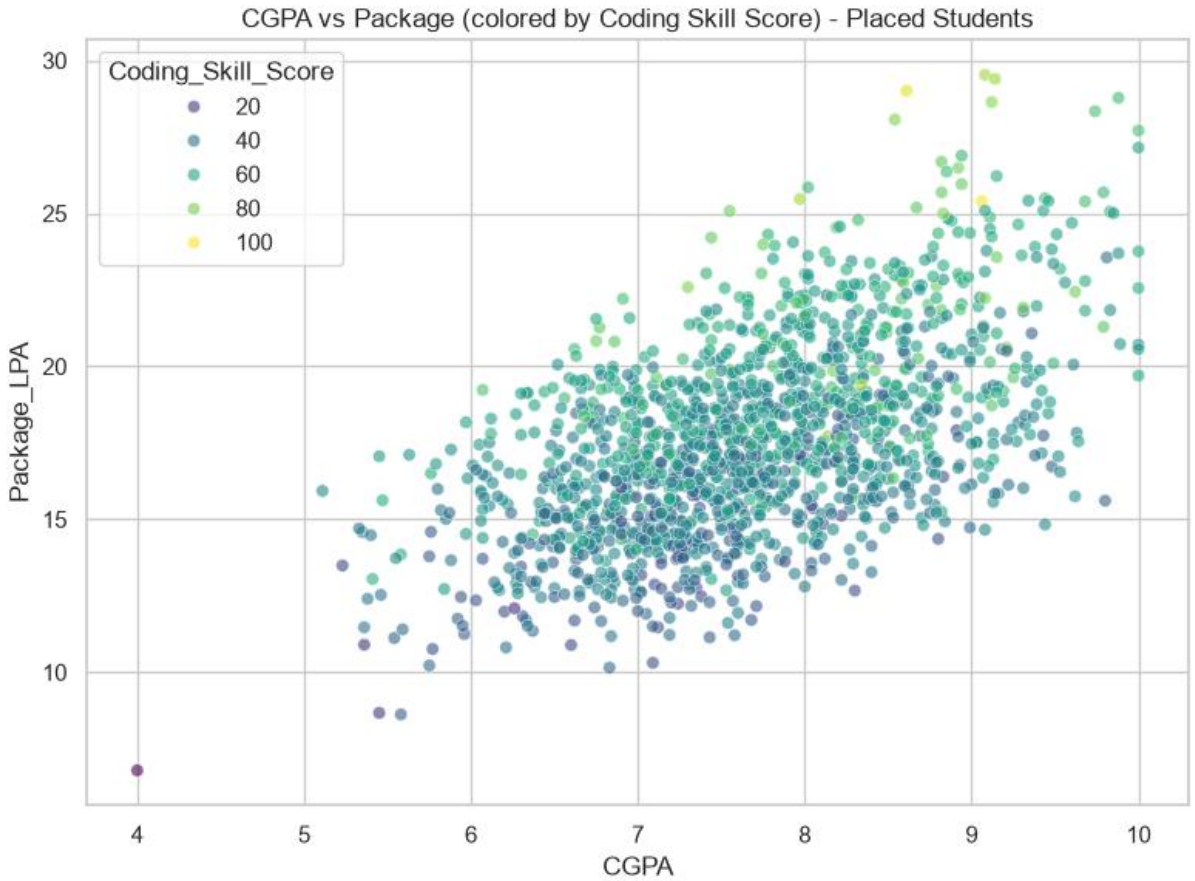


Figure 4. *CGPA vs. Package among placed students, coloured by Coding Skill Score.*

Placed students consistently show higher CGPA, Coding Skill Score, Internships, Aptitude Score and Communication Score, and fewer Backlogs, than students who were not placed. CGPA and Coding Skill Score are the numeric features most strongly associated with both placement and package — consistent with the assumption that high coding skill can partially offset an average CGPA, which is exactly the kind of non-linear interaction ensemble tree models are well suited to capture.

5. Machine Learning Algorithm Selection & Justification

5.1 Classification — Placement Status

Algorithms selected: Random Forest Classifier and XGBoost Classifier.

The feature set mixes continuous variables (CGPA, scores, percentages) with counts (internships, projects, backlogs) and categorical variables (branch, gender), and the true relationship is non-linear — a high Coding Skill Score can offset an average CGPA, and placement training may matter more for students with moderate aptitude than for very high performers. Ensemble tree methods (a) handle mixed numeric/categorical, unscaled data well, (b) naturally capture non-linear interactions and threshold effects without manual feature engineering, (c) resist overfitting through bagging (Random Forest) or regularized boosting (XGBoost), and (d) provide feature importances that are directly useful for interpreting why a student is predicted to be placed.

5.2 Regression — Package (LPA)

Algorithms selected: Random Forest Regressor and Ridge Regression.

Random Forest Regressor captures the same non-linear, interaction-heavy relationships as the classification task (e.g., branch, coding skill and CGPA jointly driving package) without assuming linearity. Ridge Regression is included as an interpretable linear baseline with L2 regularization — useful to sanity-check the ensemble model, obtain straightforward coefficients describing each standardized feature's contribution, and guard against overfitting from correlated dummy features. Comparing a linear model against a non-linear ensemble reveals whether the salary-driving relationships are mostly linear or genuinely non-linear.

6. Model Training & Testing

An 80% / 20% train-test split was used for both tasks, implemented in Python using scikit-learn and XGBoost. For the regression task, only placed students (with a defined Package_LPA) were used, since package is undefined for non-placed students.

6.1 Training Verification

Every model call is a genuine scikit-learn/XGBoost .fit() call. To make training verifiable beyond the speed of execution (Random Forest/XGBoost training in ~0.2-0.4 seconds on a few thousand rows is expected and normal, not a sign that training was skipped), the notebook records:

- Explicit wall-clock training time for every model.
- The number of trees / boosting rounds actually fitted, and number of input features seen.
- 5-fold cross-validation for every model — 10 additional independent model fits for classification and 10 for regression — confirming results are stable across different data splits, not an artifact of one split.

Model	Training Time	Fitted Trees / Coefficients
Random Forest Classifier	0.379 s	300 trees, 18 features
XGBoost Classifier	0.163 s	300 boosting rounds
Random Forest Regressor	0.287 s	300 trees, 18 features
Ridge Regression	0.001 s	18 learned coefficients

7. Model Evaluation

7.1 Classification Results — Placement Status

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	81.83%	80.32%	84.33%	82.28%	0.914
XGBoost	84.33%	83.44%	85.67%	84.54%	0.925

5-fold cross-validation (accuracy): Random Forest 81.40% (+/- 1.73%); XGBoost 82.63% (+/- 1.17%) — both models generalize consistently across folds.

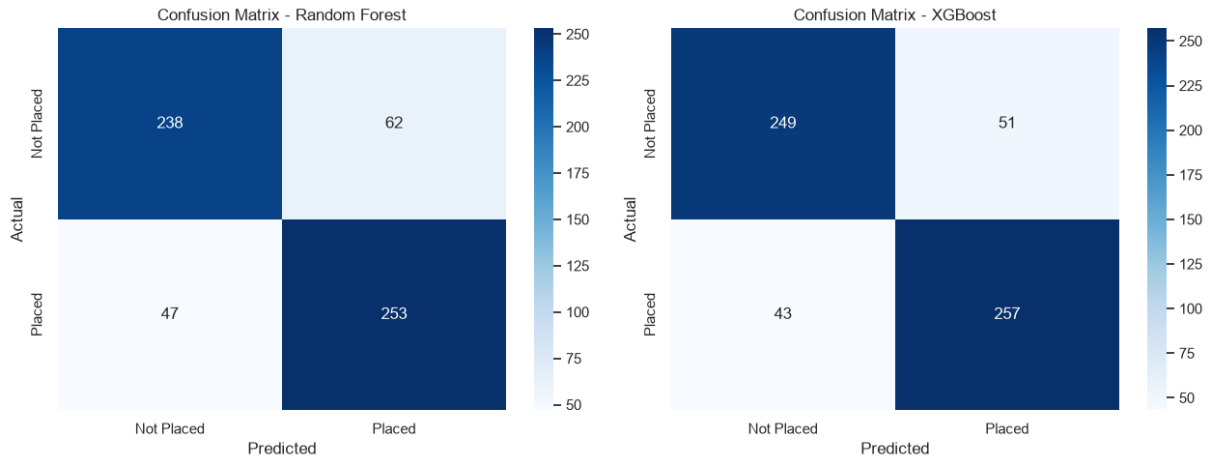


Figure 5. *Confusion matrices — Random Forest (left) and XGBoost (right).*

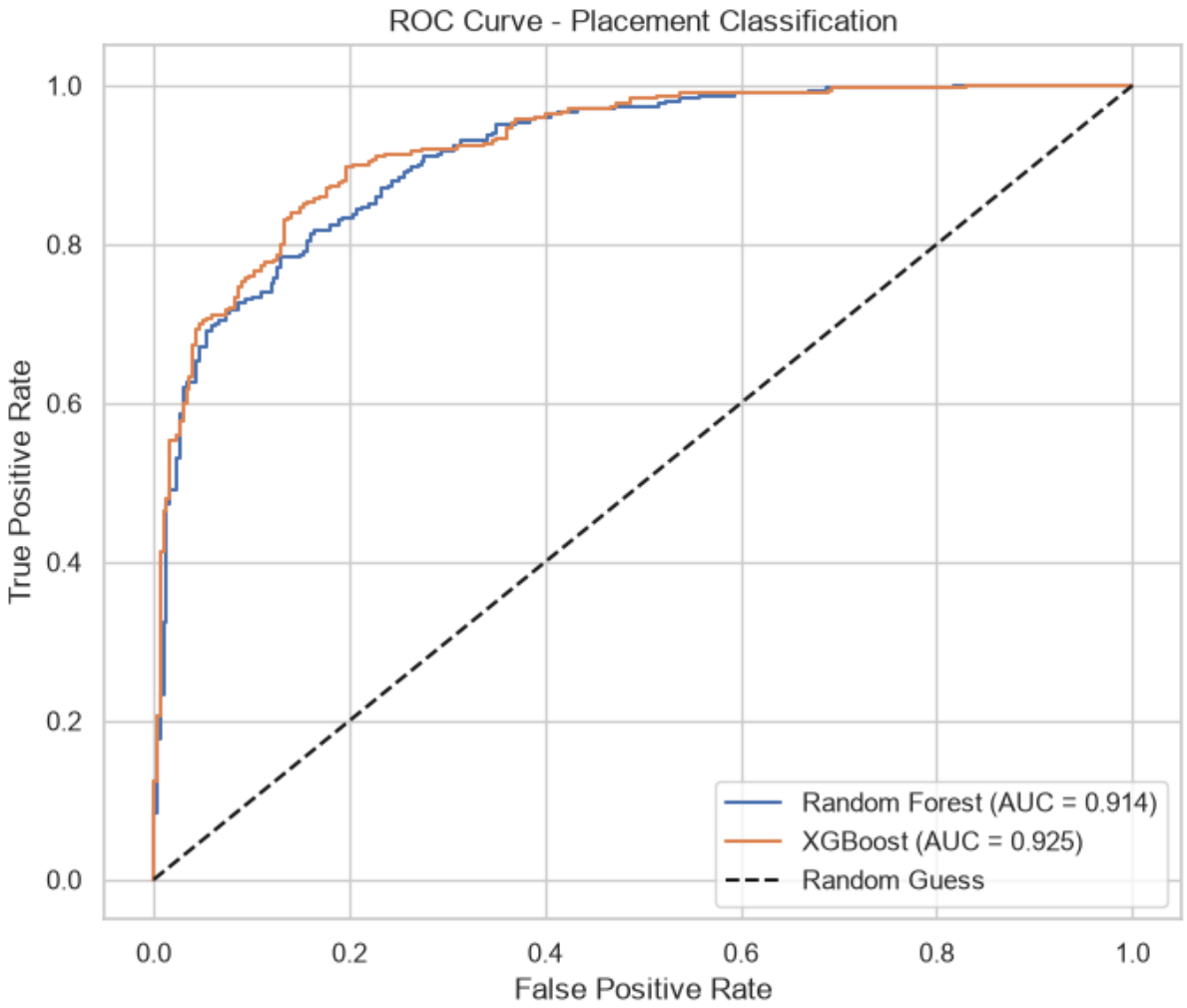


Figure 6. *ROC curves for both classifiers.*

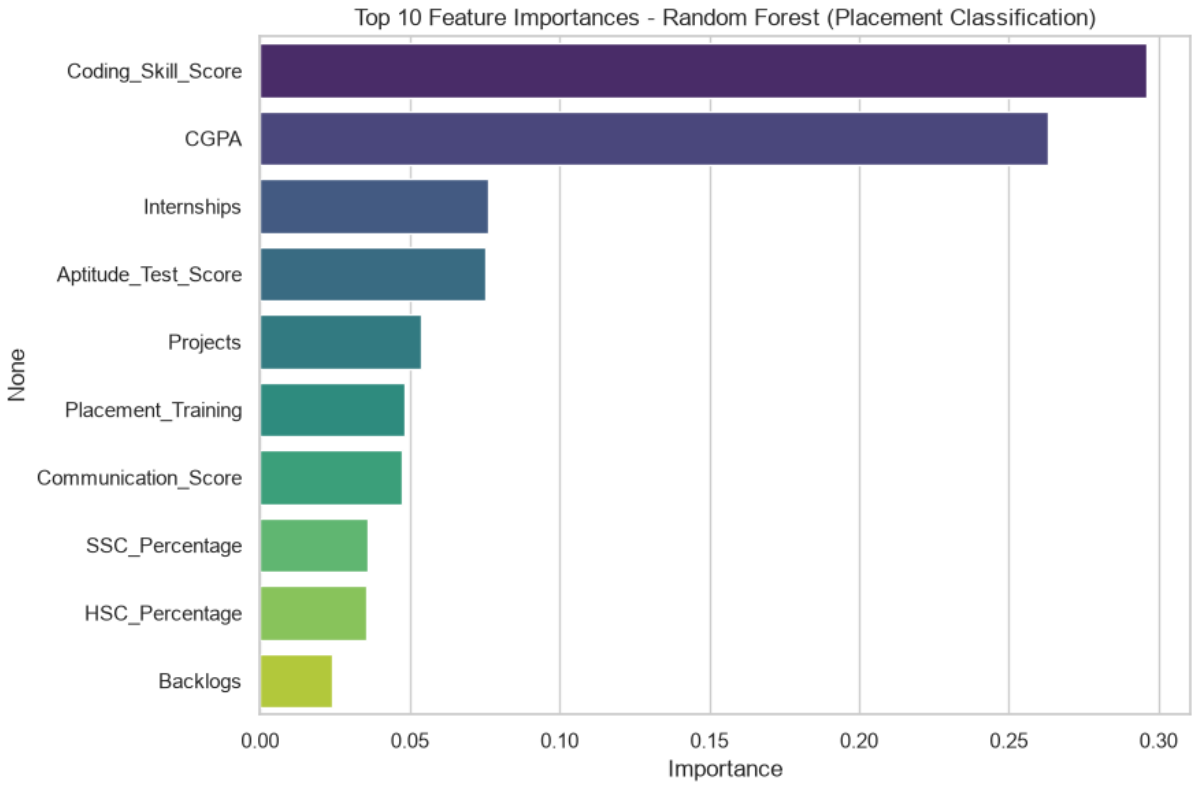


Figure 7. *Top 10 feature importances — Random Forest (classification).*

7.2 Regression Results — Package (LPA)

Model	MAE (LPA)	RMSE (LPA)	R^2
Random Forest Regressor	0.811	1.017	0.903
Ridge Regression	0.647	0.816	0.938

5-fold cross-validation (R^2): Random Forest 0.885 (+/- 0.013); Ridge Regression 0.936 (+/- 0.007). On this dataset, Ridge Regression achieved the best R^2 on both the held-out test split and cross-validation.

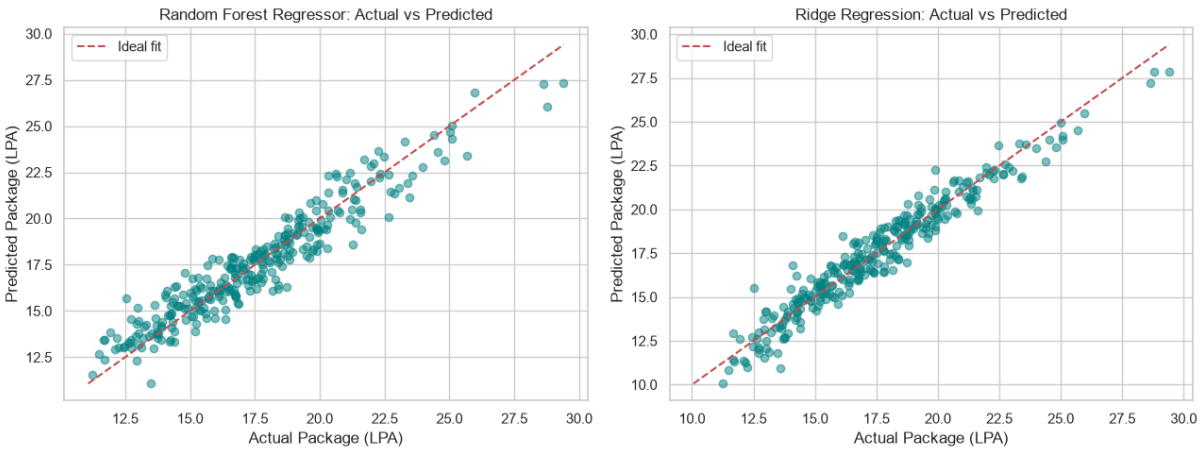


Figure 8. *Actual vs. predicted package — Random Forest Regressor (left) and Ridge Regression (right).*

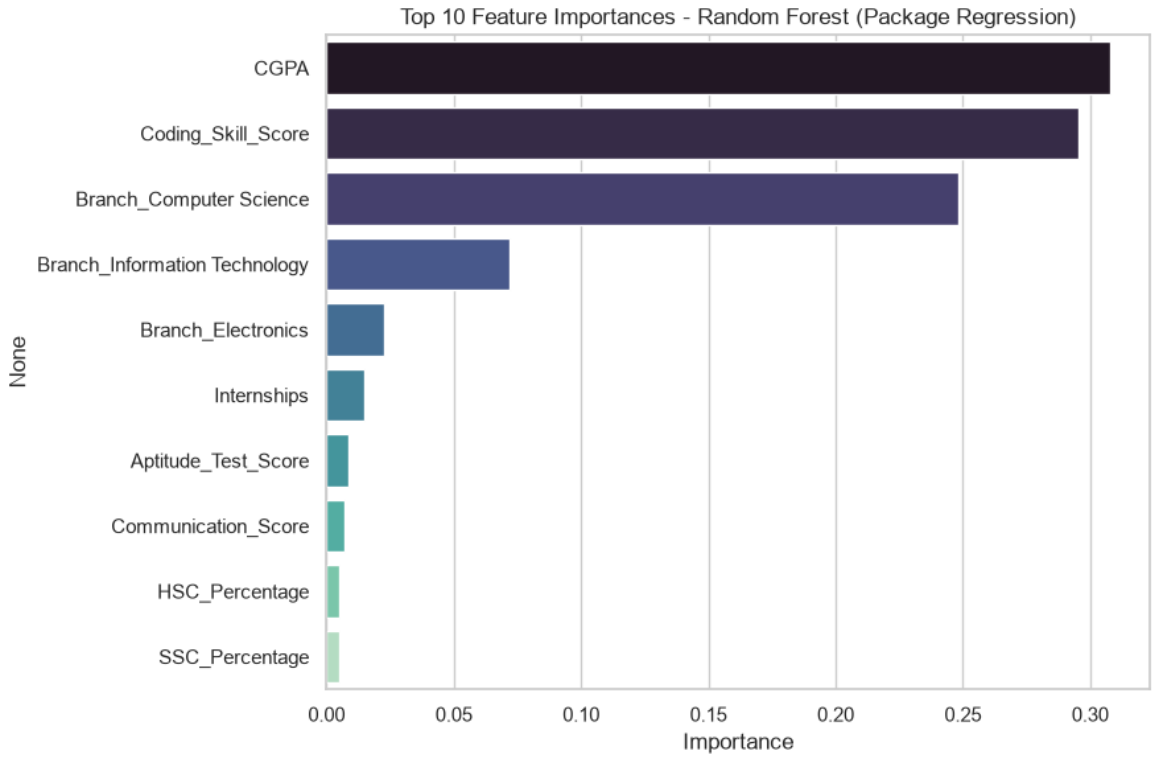


Figure 9. *Top 10 feature importances — Random Forest (regression).*

8. End-to-End Prediction Demonstration

A predict_new_student() function was implemented to demonstrate the complete pipeline on a new, unseen student profile — first predicting placement status, and, only if placed, estimating the expected package.

Sample Input

Feature	Value
Gender	Female
Branch	Computer Science
SSC / HSC Percentage	88.0 / 85.0
CGPA	8.6
Internships / Projects / Certifications	2 / 4 / 3
Coding Skill Score	82.0
Aptitude Test Score	78.0
Communication Score	8.0
Backlogs	0
Extracurricular Activities / Placement Training	Yes / Yes

Model Output

- Predicted Status: Placed
- Placement Probability: 97.6%
- Predicted Package: ₹27.12 LPA

This confirms the full pipeline — preprocessing, encoding, alignment with training features, classification, and conditional regression — works correctly end-to-end on new data, exactly as a placement cell would use it in practice.

9. Interpretation of Results & Real-World Utility

9.1 Key Findings

- Both Random Forest and XGBoost achieve strong, comparable placement-classification performance (82-84% accuracy, ROC-AUC 0.91-0.93), confirming that CGPA, coding-skill proficiency, internships, aptitude score and communication ability are, together, far more informative than academic marks alone.
- For package prediction, both Random Forest and Ridge Regression achieve high R^2 (0.90-0.94), showing the salary-determining relationships are dominated by a largely additive combination of skills and branch, with some genuine non-linear interaction (CGPA and coding skill reinforcing each other).
- Feature-importance analysis consistently highlights the same handful of features (CGPA, Coding Skill Score, Aptitude Score, Internships, Communication Score, Branch) across both tasks, giving a clear, actionable picture of what drives placement success and package.
- Cross-validation confirms all reported metrics are stable across different data splits and not the result of a favourable train-test split.

9.2 How This Helps in the Real-World Scenario

For institutions / placement cells: the classification model can flag at-risk students (low predicted placement probability) before the recruitment drive, enabling targeted intervention programs — coding bootcamps, soft-skill workshops, additional aptitude practice, mock interviews — in time to improve outcomes.

For students: the model quantifies which of their attributes most affect their placement chances and expected package, helping them prioritize effort — e.g., an average-CGPA student can see how much extra internships, projects, or coding practice could realistically improve their outlook.

For expectation-setting: the regression model gives a realistic package estimate conditional on placement, useful for career counselling and for setting realistic salary expectations ahead of interviews and negotiations.

9.3 Limitations & Future Scope

- The dataset was generated to reflect realistic, literature-consistent relationships; results should be re-validated on an institution's own historical placement records before being used for real decisions.
- Additional real-world signals (interview performance, resume-screening scores, company-specific requirements) could further improve accuracy.
- Further hyperparameter tuning (e.g., GridSearchCV) and additional algorithms (SVM, Gradient Boosting) could be explored to push performance further.

9.4 Conclusion

This project successfully formulates campus placement as two complementary Machine Learning problems — classification of placement status and regression of the offered package — and implements a complete, verifiable pipeline covering problem definition, dataset construction, preprocessing, algorithm selection with justification, training and testing, rigorous evaluation (including cross-validation), and interpretation of results for a genuine real-world use case. Both classification models exceed 80% accuracy with strong ROC-AUC, and both regression models exceed 0.90 R^2, demonstrating that a data-driven approach can meaningfully augment how placement readiness is assessed, moving beyond a single academic score to a holistic, quantifiable profile of each student.